



UNIVERSIDAD ESTATAL AMAZÓNICA

SEMANA:

2

TAREA:

**MÉTODOS DE ORGANIZACIÓN Y ACCESO A DATOS EN
SISTEMAS DE FICHEROS**

MATERIA:

ADMINISTRACION DE SISTEMAS OPERATIVOS

PARALELO:

D

ESTUDIANTE:

MINAYA GARCIA JEFFERSON CRISTOPHER

PERIODO

2026 – 2026

1. Introducción

Los sistemas de ficheros son el mecanismo fundamental mediante el cual un sistema operativo organiza, almacena y recupera datos en dispositivos de almacenamiento. La elección del método de acceso a los archivos determina directamente el rendimiento, la eficiencia y la adecuación de un sistema para un propósito específico.

Existen tres métodos principales de acceso a datos en sistemas de ficheros, cada uno con características, ventajas y casos de uso diferenciados. El presente documento analiza cada uno de estos métodos a través de ejemplos reales y representativos tomados de sistemas, aplicaciones y tecnologías ampliamente utilizados en entornos profesionales y de producción.

Método	Característica principal	Ejemplo analizado	Complejidad
Acceso Secuencial	Lectura registro por registro en orden	Apache Web Server — access.log	$O(n)$
Acceso Directo	Salto inmediato a cualquier posición	SSD NVMe con NTFS (Windows)	$O(1)$
Acceso Indexado	Búsqueda por clave en estructura de índice	MySQL InnoDB — Árbol B+	$O(\log n)$

2. Método 1 — Acceso Secuencial

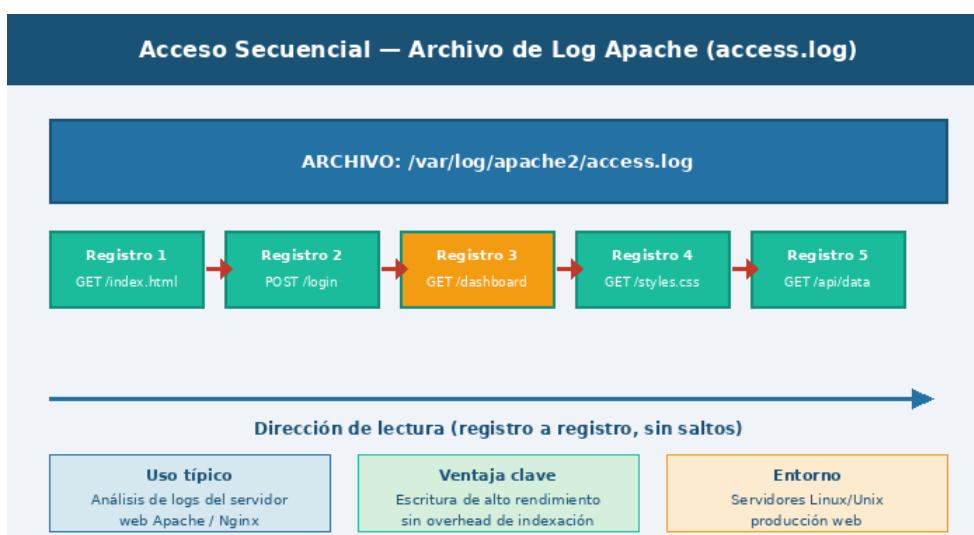
Sistema: Servidor Web Apache HTTP Server — Archivo access.log

2.1 Descripción del sistema y tecnología

Apache HTTP Server es uno de los servidores web más utilizados en el mundo, con una cuota de mercado superior al 30% de los servidores web activos en Internet. Su mecanismo de registro de eventos (logging) es un ejemplo paradigmático del acceso secuencial aplicado en producción.

Nombre del sistema	Apache HTTP Server 2.4 / Nginx
Archivo de datos	/var/log/apache2/access.log
Tipo de acceso	Secuencial — escritura al final, lectura de inicio a fin
Entorno típico	Servidores Linux/Unix en producción web, centros de datos
Tamaño típico	Desde megabytes hasta cientos de gigabytes diarios

2.2 Diagrama del método de acceso



2.3 Forma en que organiza y accede a los datos

El archivo access.log de Apache implementa el patrón de acceso secuencial de la siguiente manera: cada petición HTTP que llega al servidor genera automáticamente una nueva línea de texto que se anexa al final del archivo. Esta escritura siempre ocurre al final, nunca en medio ni al principio, lo que elimina por completo la necesidad de reorganización o indexación.

Durante el análisis posterior (post-procesamiento) de los logs, herramientas como GoAccess, AWStats o scripts personalizados en Python/Bash leen el archivo desde el primer registro hasta el último, procesando cada línea en orden. Ningún registro es saltado; todos son leídos secuencialmente.

Cada línea contiene: IP de origen, timestamp, método HTTP, recurso solicitado, código de respuesta y bytes transferidos.

La escritura es atómica: cada registro se escribe completo antes del siguiente.

La rotación de logs (logrotate) crea nuevos archivos periódicamente sin alterar el acceso secuencial.

2.4 Justificación de la idoneidad del método

Eficiencia en escritura: Dado que cada registro simplemente se añade al final del archivo, la operación de escritura tiene un tiempo constante $O(1)$ y no requiere calcular posiciones ni actualizar índices. Esto permite que Apache procese miles de peticiones por segundo sin que el logging sea un cuello de botella.

Tipo de acceso requerido: El análisis de logs requiere procesar todos los registros sin excepción (para calcular estadísticas globales), lo que hace que el acceso secuencial sea ideal: no se desperdicia ninguna lectura.

Ventajas frente a otros métodos: Un acceso directo o indexado sería innecesariamente complejo y consumiría recursos para mantener estructuras de índice en tiempo real durante la escritura. El acceso secuencial es más simple, más rápido para escritura continua y más tolerante a fallos del sistema (un fallo no corrompe el índice).

Ventaja clave: Escritura $O(1)$ sin overhead — ideal para generación masiva de registros en tiempo real

3. Método 2 — Acceso Directo (Aleatorio)

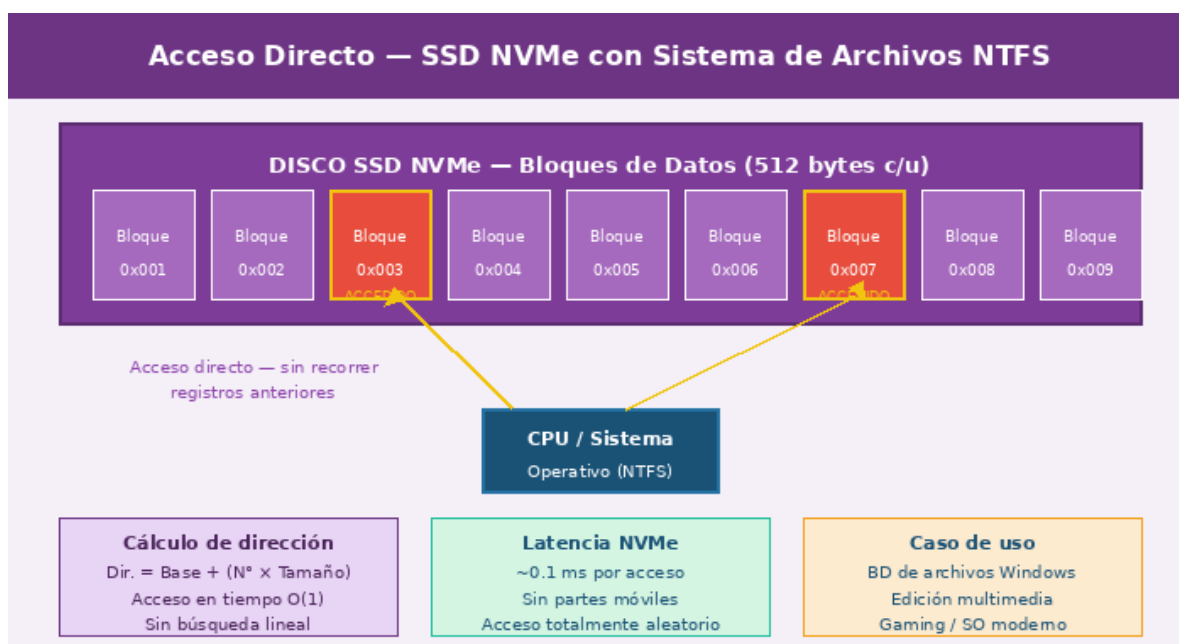
Sistema: Disco SSD NVMe con Sistema de Archivos NTFS (Windows 10/11)

3.1 Descripción del sistema y tecnología

NTFS (New Technology File System) es el sistema de archivos principal de Microsoft Windows, implementado desde Windows NT en 1993 y usado en la actualidad en más del 70% de los sistemas de escritorio y servidores Windows. Cuando NTFS opera sobre un SSD NVMe (Non-Volatile Memory Express), el acceso directo a cualquier bloque del disco se realiza con latencias inferiores a 0.1 milisegundos.

Nombre del sistema	Microsoft NTFS sobre SSD NVMe (PCIe Gen 4/5)
Tipo de acceso	Directo (aleatorio) — acceso inmediato a cualquier bloque
Mecanismo	Cálculo aritmético: $\text{Dir.} = \text{Base} + (\text{N}^\circ \text{ bloque} \times \text{Tamaño de bloque})$
Entorno típico	Estaciones de trabajo, servidores Windows, gaming, edición multimedia
Latencia típica	~0.05 - 0.2 ms (NVMe PCIe Gen4) vs. ~5 ms en HDD

3.2 Diagrama del método de acceso



3.3 Forma en que organiza y accede a los datos

NTFS organiza el disco en unidades denominadas clústeres (cluster), típicamente de 4 KB a 64 KB. La MFT (Master File Table) actúa como base de datos central que almacena la ubicación exacta de cada archivo en el disco. Cuando se solicita acceder a un archivo o un bloque específico, el sistema operativo consulta la MFT para obtener la dirección física exacta y se desplaza directamente a esa posición sin recorrer otros bloques.

La fórmula de acceso directo es: Dirección física = Dirección base + (Número de bloque × Tamaño del bloque). Esta operación matemática de tiempo constante O(1) permite al controlador NVMe enviar el comando de lectura/escritura directamente al bloque de memoria NAND Flash requerido.

El SSD NVMe no tiene partes mecánicas móviles: puede acceder a cualquier celda de memoria simultáneamente.

NTFS admite fragmentación: un archivo puede distribuirse en múltiples bloques no contiguos, y cada bloque se accede directamente.

Windows utiliza caché de disco para acelerar accesos repetidos al mismo bloque.

3.4 Justificación de la idoneidad del método

Eficiencia en lectura y escritura: A diferencia del acceso secuencial, el método directo permite que el sistema operativo acceda inmediatamente al bloque que necesita, sin leer los anteriores. Esto es crítico en aplicaciones como bases de datos, edición de video 4K o

videojuegos con mundos abiertos, donde se necesitan distintos sectores de datos simultáneamente.

Tipo de acceso requerido: En Windows, el usuario puede abrir cualquier archivo, saltar a cualquier posición dentro de él (seek), editar una sección específica y guardarlo. Ninguna de estas operaciones requiere recorrer el archivo desde el inicio.

Ventajas frente a otros métodos: El acceso secuencial sería inaceptablemente lento para editar un archivo de video de 100 GB en el minuto 45: habría que leer los primeros 44 minutos antes de llegar al punto deseado. El acceso directo elimina este problema totalmente.

Ventaja clave: Tiempo de acceso $O(1)$ — posicionamiento inmediato sin recorrido lineal previo

4. Método 3 — Acceso Indexado

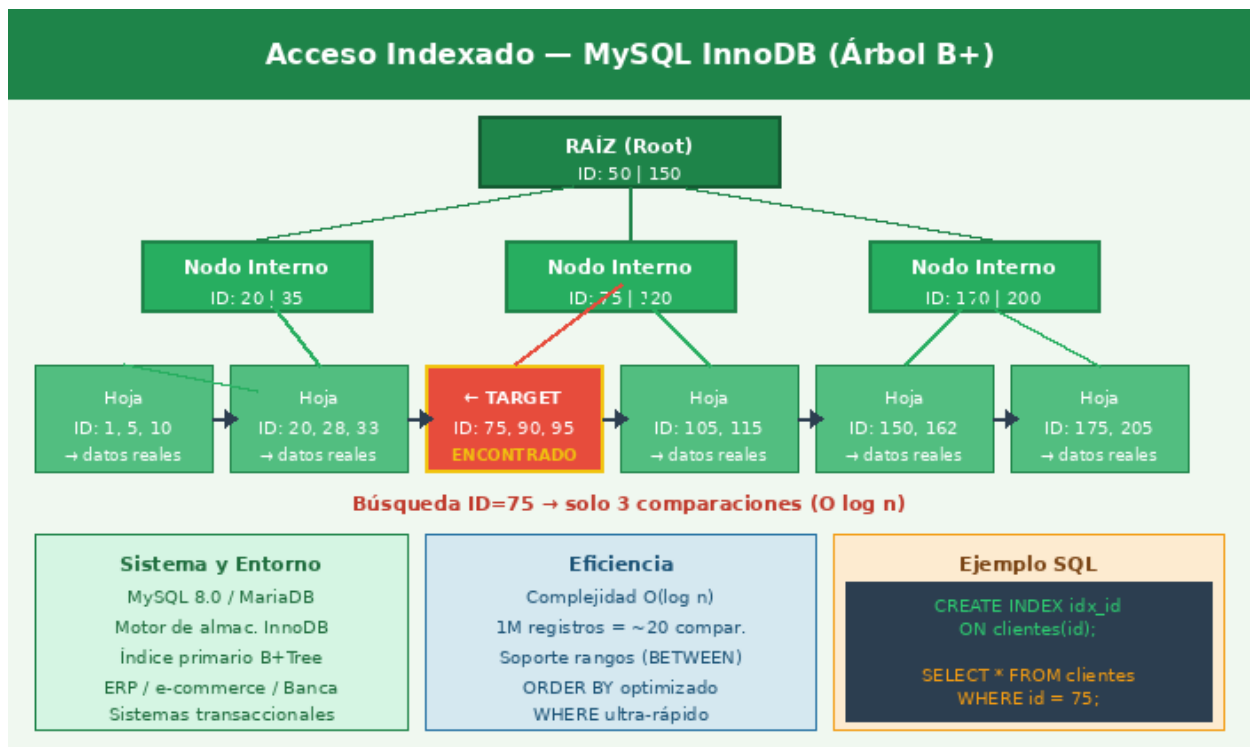
Sistema: MySQL 8.0 con Motor InnoDB — Índices de tipo Árbol B+

4.1 Descripción del sistema y tecnología

MySQL es el sistema de gestión de bases de datos relacional (RDBMS) de código abierto más utilizado globalmente, presente en plataformas como WordPress, Facebook (versiones históricas), Twitter y miles de aplicaciones empresariales. Su motor de almacenamiento predeterminado, InnoDB, implementa el acceso indexado mediante una estructura de datos denominada Árbol B+ (B+ Tree), que permite localizar cualquier registro por su clave en tiempo logarítmico $O(\log n)$.

Nombre del sistema	MySQL 8.0 / MariaDB 11.x — Motor InnoDB
Tipo de acceso	Indexado — búsqueda por clave en árbol B+
Estructura de índice	Árbol B+ balanceado (nodos hoja encadenados para rangos)
Entorno típico	ERP empresarial, e-commerce, banca, sistemas SaaS, plataformas web
Eficiencia	1 millón de registros → máximo 20 comparaciones para encontrar cualquier valor

4.2 Diagrama del método de acceso



4.3 Forma en que organiza y accede a los datos

InnoDB organiza físicamente los datos de cada tabla en una estructura de árbol B+ denominada "índice agrupado" (clustered index). Esto significa que las filas de datos no se almacenan en un heap desordenado, sino ordenadas físicamente por la clave primaria (PRIMARY KEY) dentro de la estructura del árbol.

El árbol B+ tiene tres niveles principales: la raíz (root node), nodos internos intermedios y nodos hoja (leaf nodes). Los nodos hoja contienen los datos reales de las filas y están encadenados entre sí mediante punteros, lo que permite búsquedas por rango eficientes además de las búsquedas por clave exacta.

Cuando se ejecuta una consulta SQL como `SELECT * FROM clientes WHERE id = 75`, InnoDB no recorre todos los millones de registros. En su lugar, comienza en la raíz, compara la clave buscada con los valores del nodo, se desplaza al subárbol correspondiente, y repite hasta llegar al nodo hoja correcto. Con 1.000.000 de registros, esto requiere como máximo 20 comparaciones ($\log_2$ de un millón ≈ 20).

Índice primario: almacena los datos directamente en las hojas del árbol (clustered index).

Índices secundarios: árboles B+ adicionales que mapean claves alternativas (email, nombre) al ID primario.

Los nodos hoja encadenados permiten eficientes consultas de rango: BETWEEN, >, <, ORDER BY.



El árbol se rebalancea automáticamente al insertar o eliminar registros para mantener $O(\log n)$.

4.4 Justificación de la idoneidad del método

Eficiencia en lectura y escritura: El árbol B+ proporciona búsquedas $O(\log n)$ tanto para claves exactas como para rangos. Las inserciones y eliminaciones también son $O(\log n)$, con rebalanceo automático que garantiza que el árbol nunca degenera en una lista lineal. Esta eficiencia es crítica en sistemas transaccionales donde se realizan millones de consultas diarias.

Tipo de acceso requerido: Las bases de datos relacionales requieren acceso por clave (`WHERE id = X`), por rango (`WHERE fecha BETWEEN A AND B`), y con ordenamiento (`ORDER BY precio`). El acceso indexado mediante árbol B+ satisface todos estos patrones de consulta con alta eficiencia.

Ventajas frente a otros métodos: El acceso secuencial sería inaceptable en una tabla de 10 millones de clientes: encontrar un registro requeriría leer hasta 10 millones de filas. El acceso directo por posición numérica tampoco es útil cuando se busca por valores como email o nombre. El índice B+ es la solución óptima: acceso rápido por clave sin necesidad de posición física conocida.

Ventaja clave: $O(\log n)$ para búsquedas, rangos y ordenación — fundamental en bases de datos transaccionales

5. Tabla Comparativa de los Métodos de Acceso

La siguiente tabla resume y compara los tres métodos de acceso analizados, permitiendo visualizar de forma clara sus diferencias, ventajas y contextos de aplicación óptimos.

Criterio	Secuencial (Apache Log)	Directo (NVMe/NTFS)	Indexado (MySQL InnoDB)
Ejemplo real	Apache access.log	SSD NVMe + NTFS	MySQL InnoDB
Complejidad acceso	$O(n)$	$O(1)$	$O(\log n)$
Velocidad de escritura	Muy alta	Muy alta	Alta (con rebalanceo)
Velocidad de lectura puntual	Baja (recorre todo)	Muy alta	Alta
Búsqueda por clave	No eficiente	Solo por posición	Muy eficiente
Consultas de rango	Sí (natural)	No nativo	Muy eficiente
Overhead de estructura	Ninguno	Mínimo (MFT)	Moderado (árbol B+)
Caso de uso principal	Logs, streams, backups	Archivos, edición, OS	Bases de datos, ERP
Tolerancia a fallos	Muy alta	Alta (con journal)	Alta (ACID)

6. Conclusiones

El análisis de los tres métodos de acceso a datos en sistemas de ficheros demuestra que no existe un método universalmente superior: cada uno es óptimo para un tipo específico de necesidad de almacenamiento y recuperación de información.

El acceso secuencial es insuperable para escritura continua de alto volumen sin necesidad de recuperación aleatoria, como en sistemas de logging, telemetría o streaming de datos. Su simplicidad es su mayor fortaleza.

El acceso directo es imprescindible cuando se requiere posicionar el sistema exactamente en cualquier punto del almacenamiento en tiempo constante, como ocurre en sistemas operativos modernos con SSDs, edición multimedia y aplicaciones de usuario final.

El acceso indexado es la solución de referencia para sistemas de bases de datos donde se deben localizar registros por valor de campo, soportar múltiples criterios de búsqueda y ordenación, y mantener el rendimiento a medida que los datos crecen.

En la práctica, los sistemas modernos suelen combinar estos métodos: MySQL usa acceso directo para posicionarse en los bloques del disco mientras emplea acceso indexado para localizar los registros; Apache usa acceso secuencial para escribir los logs pero luego herramientas de análisis pueden construir índices sobre esos logs para consultas rápidas. La comprensión de cada método es, por tanto, fundamental para diseñar sistemas de información eficientes y escalables.

Referencias

Apache Software Foundation. (2024). Apache HTTP Server Documentation — Log Files. <https://httpd.apache.org/docs/2.4/logs.html>

Microsoft. (2024). How NTFS works — Windows Server. <https://learn.microsoft.com/en-us/windows-server/storage/file-server/ntfs-overview>

MySQL Documentation. (2024). InnoDB Storage Engine — Clustered and Secondary Indexes. <https://dev.mysql.com/doc/refman/8.0/en/innodb-index-types.html>

Silberschatz, A., Galvin, P., & Gagne, G. (2021). Operating System Concepts (10th ed.). Wiley.

Tanenbaum, A., & Bos, H. (2014). Modern Operating Systems (4th ed.). Pearson.